

**NYC Airbnb Listings: A Reproducible Data Workflow for the AI Programming
Foundations Project**

Lo Kai Cheung, Stanley

Woolf / Udacity — MSc in Artificial Intelligence

AI Programming Foundations — Capstone Project 1

[14 September 2026]

NYC Airbnb Listings: A Reproducible Data Workflow

Overview

This project builds a complete, reproducible data workflow using the NYC Airbnb Open Data dataset, covering ingestion, cleaning, exploratory analysis, and visualization in a single Jupyter Notebook. The dataset used is the New York City Airbnb Open Data listing set, available at <https://www.kaggle.com/dgomonov/new-york-city-airbnb-open-data>. The workflow focuses on understanding how price relates to room type, borough, and listing activity, and establishes practices that later machine learning and AI courseworks will build on.

Dataset Description

The dataset contains 48,895 Airbnb listings across New York City, with 16 columns describing each listing's host, location, room type, price, and booking activity. Key variables analyzed in this project include price (nightly rate in USD), room_type (Entire home/apt, Private room, or Shared room), neighbourhood_group (the five NYC boroughs), number_of_reviews, reviews_per_month, calculated_host_listings_count, and availability_365. Each row represents a single listing at a point in time, making the dataset a cross-sectional snapshot rather than a longitudinal record of price changes. The dataset is well suited to this project because it combines categorical, numeric, and geographic variables, allowing for a range of cleaning, grouping, and correlation techniques to be demonstrated.

Workflow Description

The notebook follows a linear workflow with five high-level stages. During ingestion, the raw CSV file is loaded with Pandas and inspected to confirm structure and data types. During cleaning, missing values, duplicate records, and price outliers are first inspected and quantified, then addressed through three documented cleaning functions. During exploratory analysis, a

reusable grouping function summarizes price by borough and by room type, and a correlation matrix is computed across the numeric variables. During visualization, three required visualizations plus one supplementary map present the price distribution by room type, average price by borough, and the correlation structure of the numeric variables. Finally, a summary section interprets the findings, states assumptions and limitations, and highlights unresolved questions.

Key Decisions and Assumptions

Cleaning choices. Missing `reviews_per_month` values (20.56% of rows) were filled with zero rather than dropped, based on the observation that these rows coincided exactly with listings missing `last_review`, indicating listings that had never been reviewed rather than a data entry problem. Rows missing critical fields (`price`, `room_type`, `neighbourhood_group`) were checked but none were found missing in this dataset. Price outliers were removed using a fixed range of \$10 to \$1,000, eliminating 11 listings priced at \$0, a data error, since no real listing is free, and 239 listings priced above \$1,000. A duplicate check found zero exact duplicate rows and zero duplicate listing IDs in this particular export, but the deduplication step was retained as a general safeguard, consistent with the principle that reproducible pipelines should not silently assume clean input data (Arevalo et al., 2022).

EDA focus. The exploratory analysis concentrated on grouping price by categorical variables, room type and borough, rather than only computing global descriptive statistics, because a single overall mean price obscures the large differences driven by property type and location. A correlation matrix was also computed across the numeric variables to check whether listing activity (`reviews`, `availability`, `host_listing_count`) related meaningfully to price before drawing conclusions from the visualizations.

Visualization design. Figure 1, a boxplot, was chosen to show the full price distribution and spread per room type, not just averages, since boxplots reveal variability that a bar chart of means would hide. Figure 2, a bar chart, was chosen for the borough comparison because it clearly ranks five discrete categories. Figure 3, a heatmap, was chosen to summarize many pairwise correlations compactly. Each figure was labeled with a title and axis labels, following documentation-based best practice for clear, reproducible visual communication (Danchev, 2022).

Results and Interpretation

Figure 1 shows that Entire home/apt listings have both the highest median price and the widest spread of the three room types, consistent with them offering full living space rather than a shared or single room; private rooms occupy a lower, tighter price band, and shared rooms are the cheapest and least variable. Figure 2 shows that Manhattan listings average roughly double the price of Bronx listings, with Brooklyn, Staten Island, and Queens falling in between, indicating that borough is one of the strongest single predictors of price in this dataset. Figure 3 shows that `number_of_reviews` and `reviews_per_month` are moderately correlated with each other, as expected, since both measure review activity, but neither correlates strongly with price, suggesting that review volume is driven more by how long a listing has been active than by its price point.

Responsible Practice (Bias and Data Quality)

Two cleaning decisions carry a risk of introducing bias. First, filling missing `reviews_per_month` values with zero assumes that missingness reflects a true absence of reviews; if any of those rows were instead the result of a data collection error, this assumption would understate review activity for those listings. Second, removing listings priced below \$10 or

above \$1,000 excludes both likely data errors and a small number of genuine luxury or promotional listings, which could bias downstream summary statistics toward the middle of the market and understate the true range of NYC Airbnb pricing. This risk is consistent with warnings in the data cleaning literature that overly aggressive filtering can remove valid extreme cases along with genuine errors (Arevalo et al., 2022). To reduce this risk further, a next step would be to examine the excluded rows individually rather than applying a single fixed threshold, and to report cleaning decisions transparently alongside the analysis, as was done in this project's dedicated cleaning-justification section.

Reproducibility

This project supports reproducibility in three ways. First, all dependencies are pinned in `requirements.txt`, generated with `pip freeze`, so another user can recreate the exact Python environment. Second, the Git and GitHub workflow tracks development through multiple commits on a dedicated development branch before merging into `main`, giving a transparent history of how the notebook evolved. Third, every cleaning and analysis step in the notebook is implemented as a documented, reusable function rather than inline one-off code, so the same pipeline can be re-run on an updated data export and produce comparable results. This approach reflects the core principle of open, reproducible data science workflows described by Danchev (2022), who argues that combining open-source tools, transparent code, and clear documentation is central to reproducible analysis.

References

- Arevalo, M., Brownstein, N. C., Whiting, J., Meade, C. D., Gwede, C. K., Vadaparampil, S. T., Tillery, K. J., Islam, J. Y., Giuliano, A. R., & Christy, S. M. (2022). Strategies and lessons learned during cleaning of data from research panel participants: Cross-sectional web-based health behavior survey study. *JMIR Formative Research*, 6(6), e35797. <https://doi.org/10.2196/35797>
- Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>
- The pandas development team. (2024). *pandas documentation*. <https://pandas.pydata.org/docs/>
- Waskom, M. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>